



Universidade Leiria • Oeste
Escola de Tecnologia e Gestão

Engenharia Informática

DEI – Departamento de Engenharia Informática

Sistemas Computacionais

Aulas Práticas – MIPS

MIPS

- As implementações do MIPS são muito usadas em sistemas embutidos (ex: consolas de jogos (ex: Sony PlayStation 2), Windows CE, routers, etc.).
- Até 2006 foram também usadas em muitos dos computadores da Silicon Graphics, Inc.
- Outras empresas: DEC, NEC, Siemens Nixdorf, etc.
- Estima-se que durante os últimos 5 anos dos anos 90 um em cada 3 processadores RISC produzidos eram implementações MIPS

MIPS

- RISC
- Pipelined processor
- WinMIPS64 – simulador Windows que demonstra o funcionamento do MIPS

MIPS (MIPS64) - Registos

- 32 registos inteiros de uso geral (GPRs - General Purpose Registers) de 64 bits (R0..R31)
 - R0 é sempre “0” e pode ser utilizado por uma função para a qual se pretenda descartar o resultado ou como fonte de dados quando é necessário o valor 0
 - R31 é utilizado implicitamente pelas instruções JAL, BLTZAL, BLTZALL, BGEZAL e BGEZALL. Pode também ser utilizado como um registo "normal"
- 32 registos de vírgula flutuante (FPRs – Floating Point Registers) de 64 bits (F0..F31)
 - Se o bit “FR” no registo “CP0 Status” tem o valor “1”, 64 bits
 - Se o bit “FR” no registo “CP0 Status” tem o valor “0”, 32 bits
- PC (Program Counter) – endereço da instrução “seguinte”
- Registos internos

MIPS (MIPS64) - Interface

WinMIPS64 - MIPS64 Processor Simulator - D:\miracl\terminal.s

File Execute Configure Window Help

Cycles

ld r4,A(r0) IF

Registers

R0=	0000000000000000	F0=	0000000.00000000
R1=	0000000000000000	F1=	0000000.00000000
R2=	0000000000000000	F2=	0000000.00000000
R3=	0000000000000000	F3=	0000000.00000000
R4=	0000000000000000	F4=	0000000.00000000
R5=	0000000000000000	F5=	0000000.00000000
R6=	0000000000000000	F6=	0000000.00000000
R7=	0000000000000000	F7=	0000000.00000000
R8=	0000000000000000	F8=	0000000.00000000
R9=	0000000000000000	F9=	0000000.00000000
R10=	0000000000000000	F10=	0000000.00000000
R11=	0000000000000000	F11=	0000000.00000000
R12=	0000000000000000	F12=	0000000.00000000
R13=	0000000000000000	F13=	0000000.00000000
R14=	0000000000000000	F14=	0000000.00000000
R15=	0000000000000000	F15=	0000000.00000000
R16=	0000000000000000	F16=	0000000.00000000
R17=	0000000000000000	F17=	0000000.00000000
R18=	0000000000000000	F18=	0000000.00000000
R19=	0000000000000000	F19=	0000000.00000000
R20=	0000000000000000	F20=	0000000.00000000
R21=	0000000000000000	F21=	0000000.00000000

Statistics

Execution

0 Cycles
0 Instructions

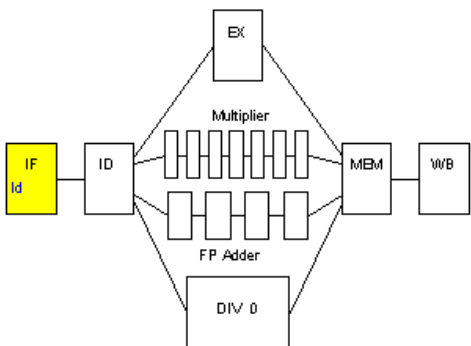
Stalls

0 RAW Stalls
0 WAW Stalls
0 WAR Stalls
0 Structural Stalls
0 Branch Taken Stalls
0 Branch Misprediction Stalls

Code size

40 Bytes

Pipeline



The diagram shows a 5-stage pipeline: IF (Instruction Fetch), ID (Instruction Decode), EX (Execute), MEM (Memory Access), and WB (Write Back). The EX stage is further detailed with sub-units: Multiplier, FP Adder, and DIV 0. Data flows from IF to ID, then to the EX stage, then to MEM, and finally to WB.

Data

```

0000 0000000000000000a A: .word 10
0008 0000000000000000 B: .word 8
0010 0000000000000000 C: .word 0
0018 00000000000010000 CR: .word32 0x10000
0020 00000000000010008 DR: .word32 0x10008
0028 0000000000000000
0030 0000000000000000
0038 0000000000000000
0040 0000000000000000
0048 0000000000000000
0050 0000000000000000
0058 0000000000000000
0060 0000000000000000
0068 0000000000000000
0070 0000000000000000
0078 0000000000000000
0080 0000000000000000
0088 0000000000000000
0090 0000000000000000
0098 0000000000000000
00a0 0000000000000000

```

Code

```

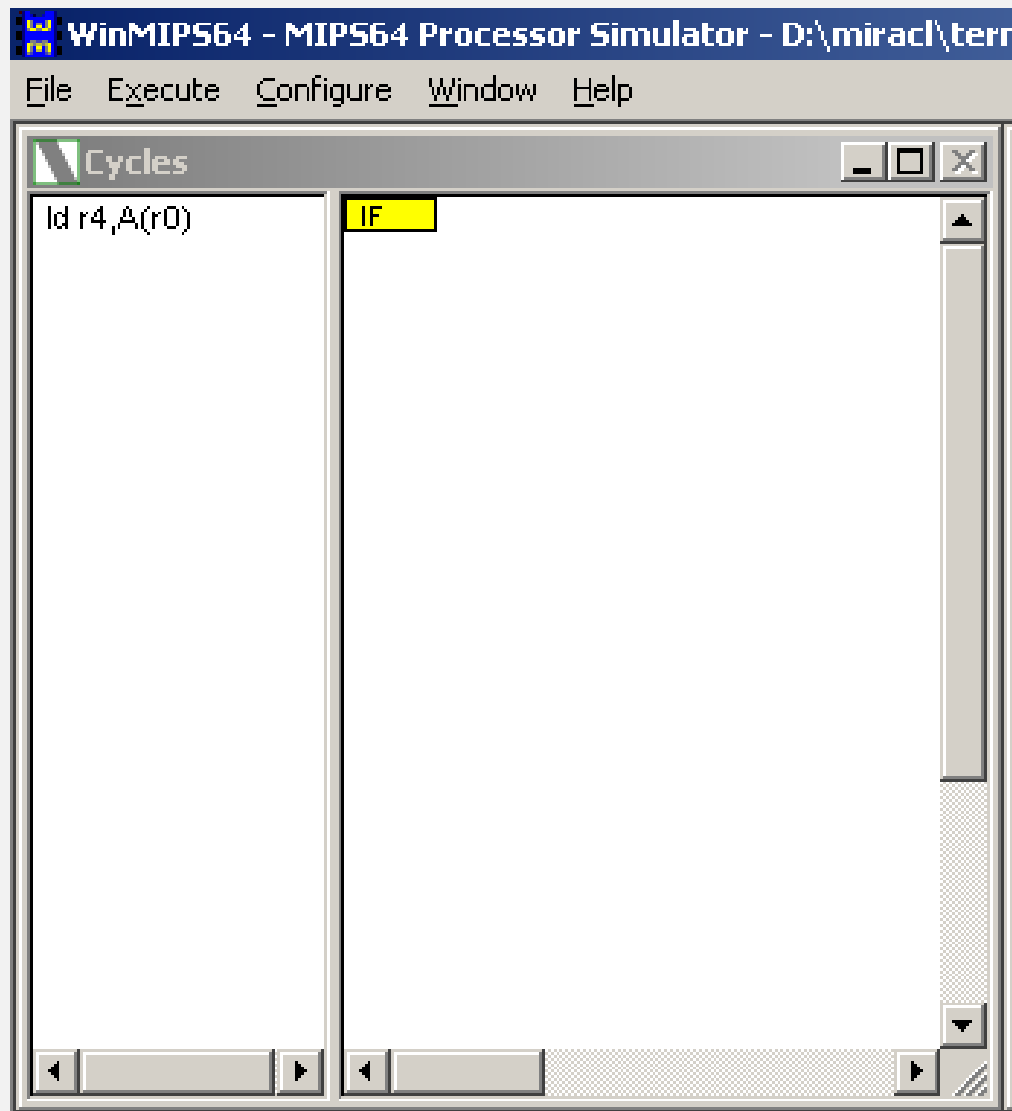
0000 dc040000 ld r4,A(r0)
0004 dc050008 ld r5,B(r0)
0008 0085182c dadd r3,r4,r5
000c fc030010 sd r3,C(r0)
0010 8c010018 lwu r1,CR(r0) ;Control F
0014 8c020020 lwu r2,DR(r0) ;Data Regi
0018 600a0001 daddi r10,r0,1
001c fc430000 sd r3,(r2) ;r3 output
0020 fc2a0000 sd r10,(r1) ;.. to scr
0024 04000000 halt
0028 00000000
002c 00000000
0030 00000000
0034 00000000
0038 00000000
003c 00000000
0040 00000000
0044 00000000
0048 00000000
004c 00000000
0050 00000000

```

Ready

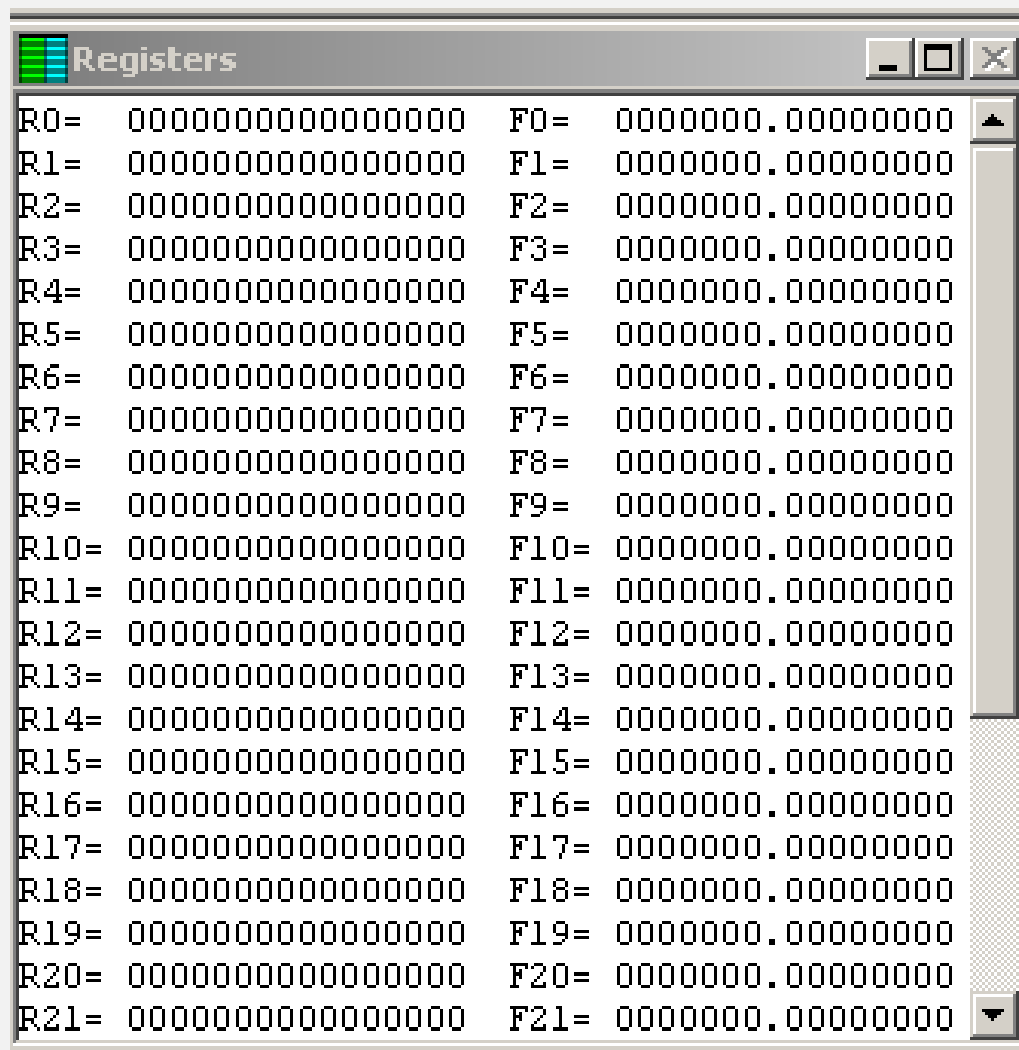
MIPS (MIPS64) - Interface

Esta janela apresenta uma disposição temporal do estado de cada instrução dentro do pipeline



MIPS (MIPS64) - Interface

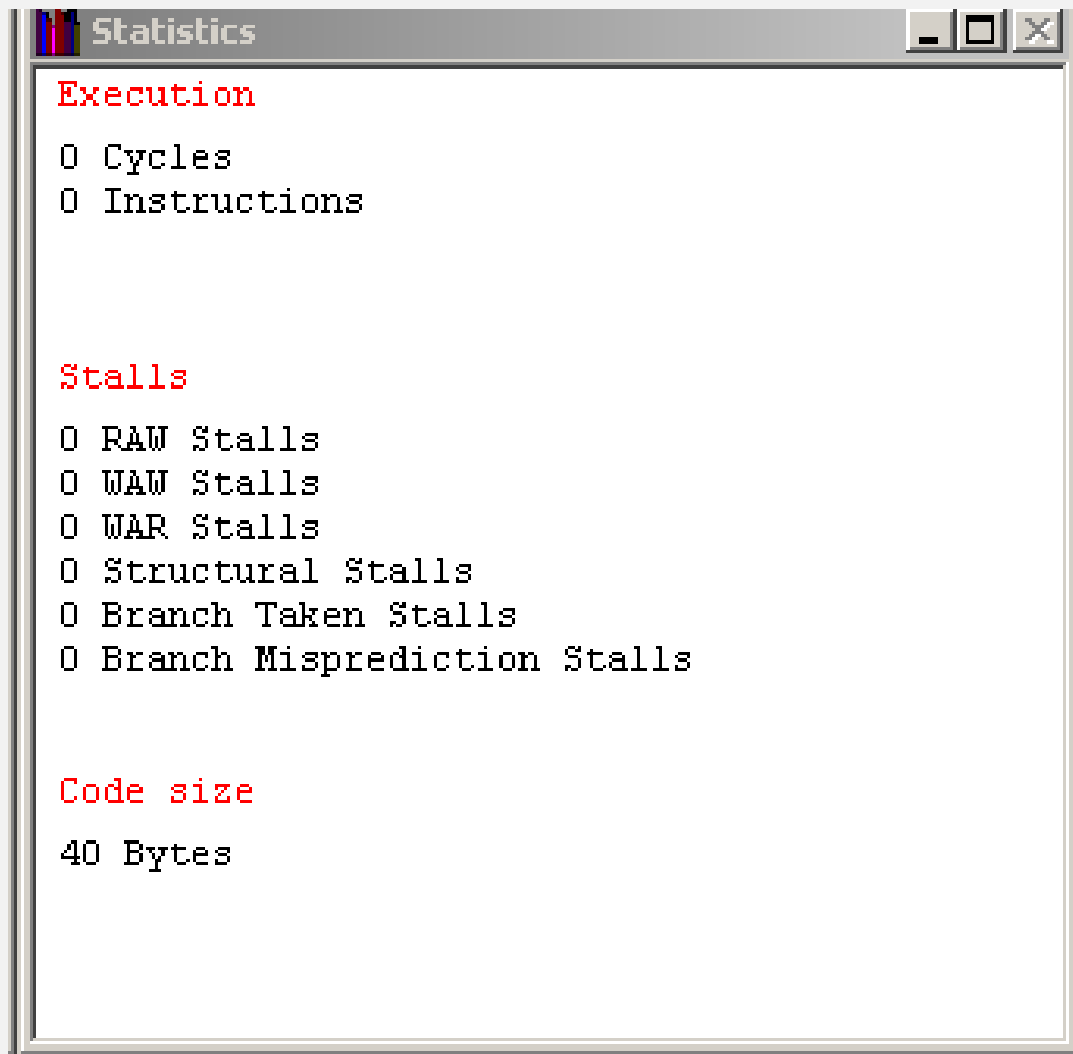
Nesta janela são apresentados os valores dos Registos (64bits). Pode aparecer a cinzento (irá ser escrito) ou com cores (representa o estado de cada instrução); Podem ser alterados



Registers			
R0=	0000000000000000	F0=	00000000.00000000
R1=	0000000000000000	F1=	00000000.00000000
R2=	0000000000000000	F2=	00000000.00000000
R3=	0000000000000000	F3=	00000000.00000000
R4=	0000000000000000	F4=	00000000.00000000
R5=	0000000000000000	F5=	00000000.00000000
R6=	0000000000000000	F6=	00000000.00000000
R7=	0000000000000000	F7=	00000000.00000000
R8=	0000000000000000	F8=	00000000.00000000
R9=	0000000000000000	F9=	00000000.00000000
R10=	0000000000000000	F10=	00000000.00000000
R11=	0000000000000000	F11=	00000000.00000000
R12=	0000000000000000	F12=	00000000.00000000
R13=	0000000000000000	F13=	00000000.00000000
R14=	0000000000000000	F14=	00000000.00000000
R15=	0000000000000000	F15=	00000000.00000000
R16=	0000000000000000	F16=	00000000.00000000
R17=	0000000000000000	F17=	00000000.00000000
R18=	0000000000000000	F18=	00000000.00000000
R19=	0000000000000000	F19=	00000000.00000000
R20=	0000000000000000	F20=	00000000.00000000
R21=	0000000000000000	F21=	00000000.00000000

MIPS (MIPS64) - Interface

Apresentação de
informação
estatística, útil
para otimização
de código



The screenshot shows a window titled "Statistics" with a standard Windows-style title bar (minimize, maximize, close buttons). The window contains a list of statistics in a monospaced font. The statistics are grouped into three sections: "Execution", "Stalls", and "Code size". Each section has a red header. The values for all metrics are currently 0, except for "Code size" which is 40 Bytes.

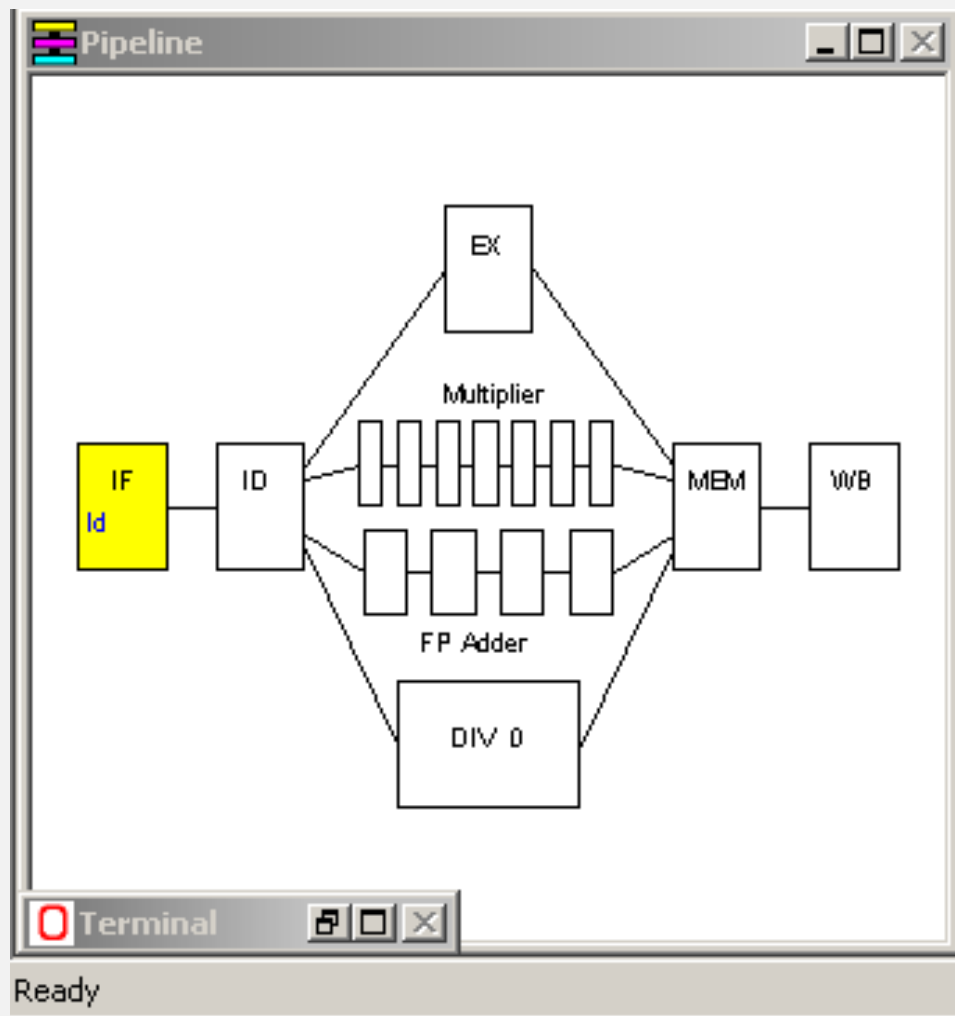
```
Statistics
Execution
0 Cycles
0 Instructions

Stalls
0 RAW Stalls
0 WAW Stalls
0 WAR Stalls
0 Structural Stalls
0 Branch Taken Stalls
0 Branch Misprediction Stalls

Code size
40 Bytes
```

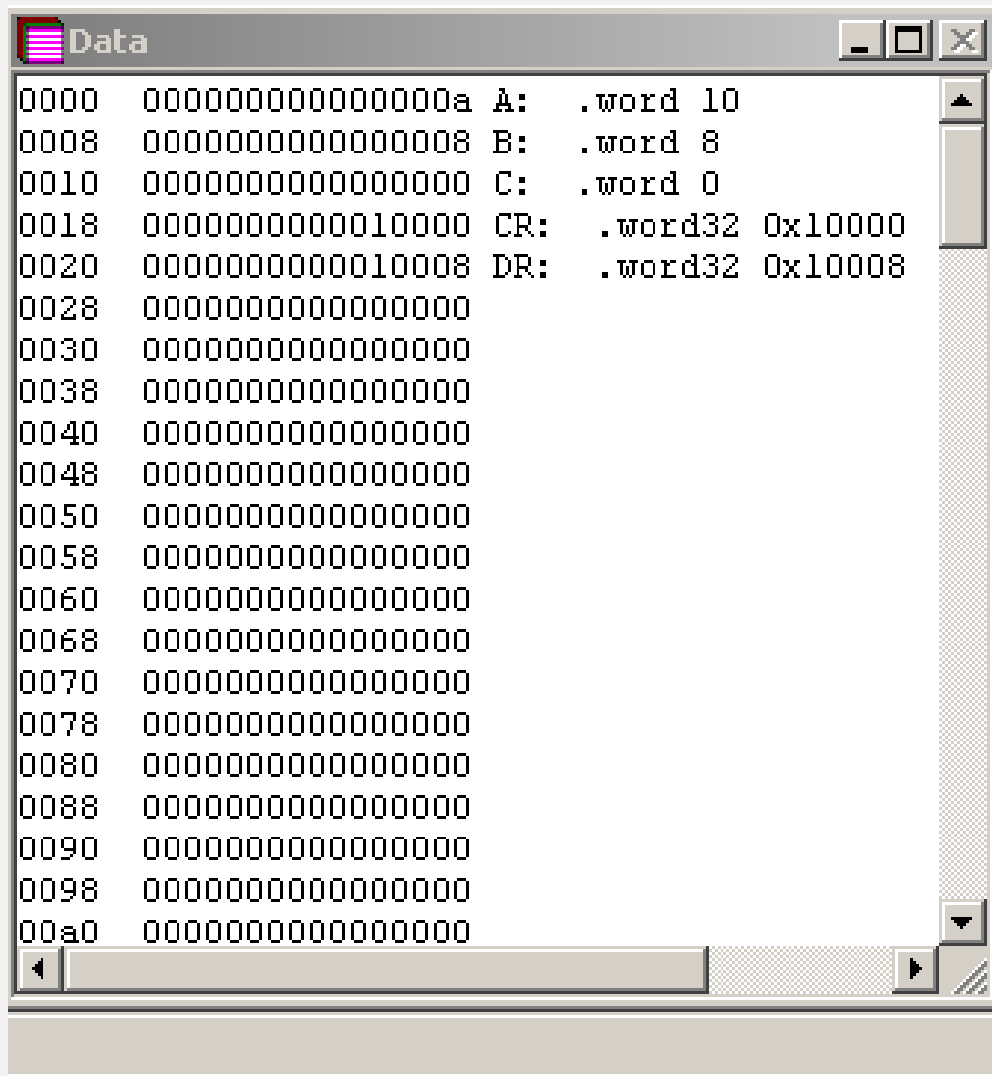
MIPS (MIPS64) - Interface

Esquema visual
do pipeline, e da
fase em que se
encontra cada
instrução



MIPS (MIPS64) - Interface

Estado da
memória; valores
iniciais e valores
atuais das
variáveis



The screenshot shows a window titled "Data" with a list of memory addresses and their corresponding values. The values are in hexadecimal. The list is as follows:

Address	Value	Label	Format
0000	0000000000000000a	A:	.word 10
0008	00000000000000008	B:	.word 8
0010	00000000000000000	C:	.word 0
0018	00000000000010000	CR:	.word32 0x10000
0020	00000000000010008	DR:	.word32 0x10008
0028	00000000000000000		
0030	00000000000000000		
0038	00000000000000000		
0040	00000000000000000		
0048	00000000000000000		
0050	00000000000000000		
0058	00000000000000000		
0060	00000000000000000		
0068	00000000000000000		
0070	00000000000000000		
0078	00000000000000000		
0080	00000000000000000		
0088	00000000000000000		
0090	00000000000000000		
0098	00000000000000000		
00a0	00000000000000000		

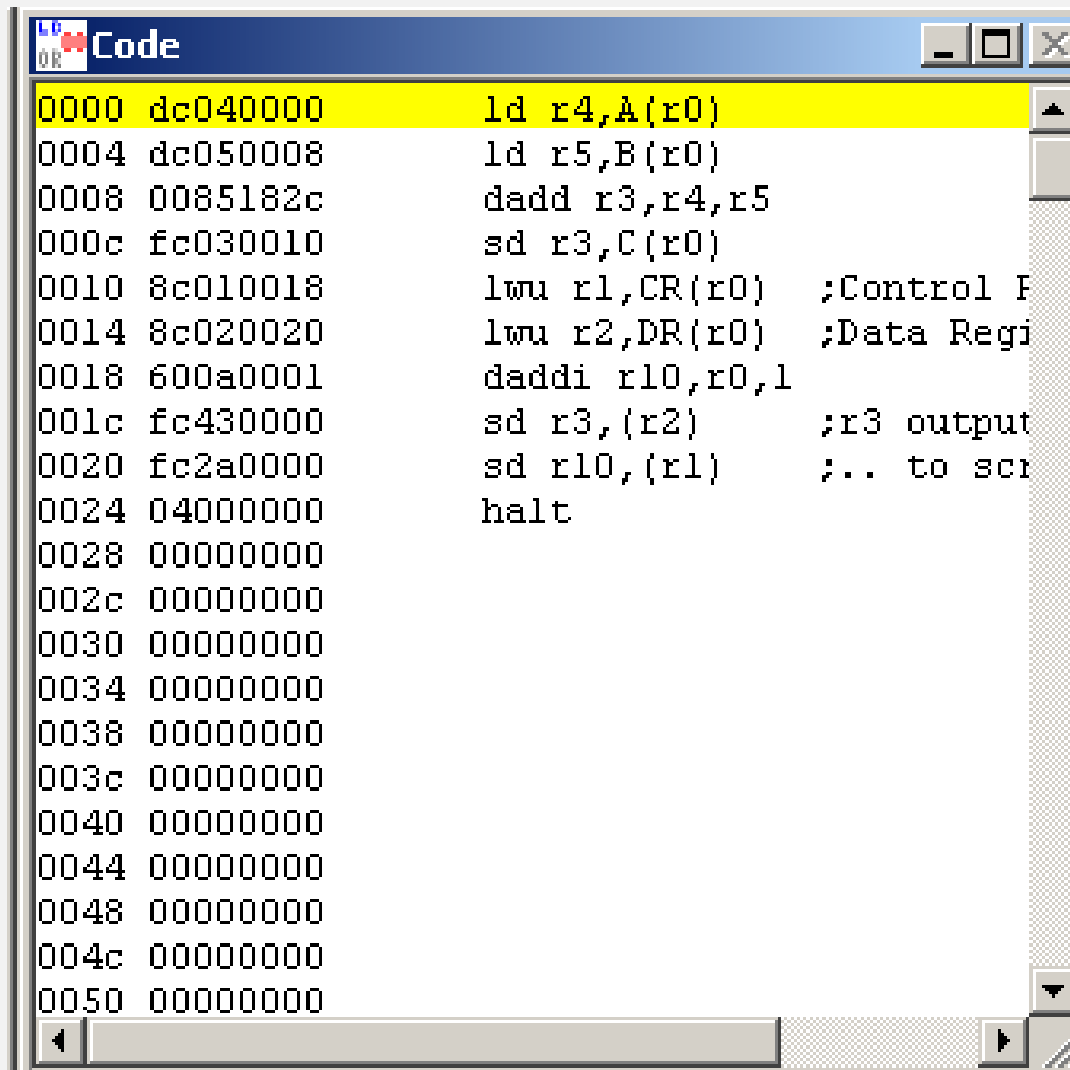
MIPS (MIPS64) - Interface

Área de Código

1) Endereço de cada instrução (4 hex.)

2) Codificação de cada instrução (8hex.)

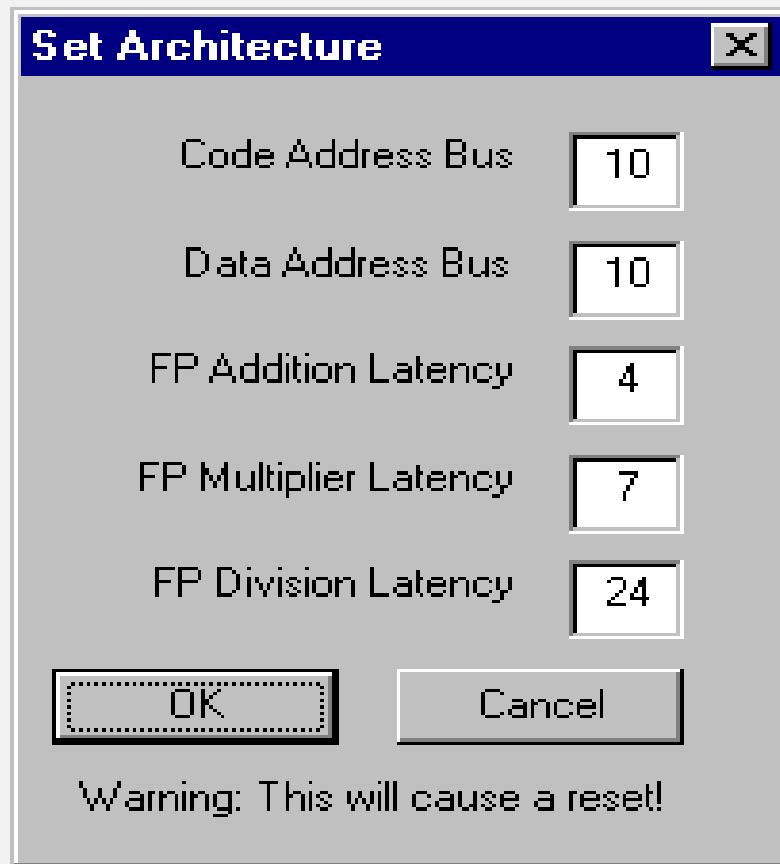
3) Código Assembler



Address	Hex Code	Assembly Instruction	Comments
0000	dc040000	ld r4,A(r0)	
0004	dc050008	ld r5,B(r0)	
0008	0085182c	dadd r3,r4,r5	
000c	fc030010	sd r3,C(r0)	
0010	8c010018	lwu r1,CR(r0)	;Control F
0014	8c020020	lwu r2,DR(r0)	;Data Regi
0018	600a0001	daddi r10,r0,1	
001c	fc430000	sd r3,(r2)	;r3 output
0020	fc2a0000	sd r10,(r1)	;.. to scr
0024	04000000	halt	
0028	00000000		
002c	00000000		
0030	00000000		
0034	00000000		
0038	00000000		
003c	00000000		
0040	00000000		
0044	00000000		
0048	00000000		
004c	00000000		
0050	00000000		

MIPS (MIPS64) - Interface

Alterações na
Arquitetura do
processador
- Menu ***Configure -
Architecture***



The image shows a 'Set Architecture' dialog box with a blue title bar and a close button (X). It contains five settings, each with a label and a numeric input field:

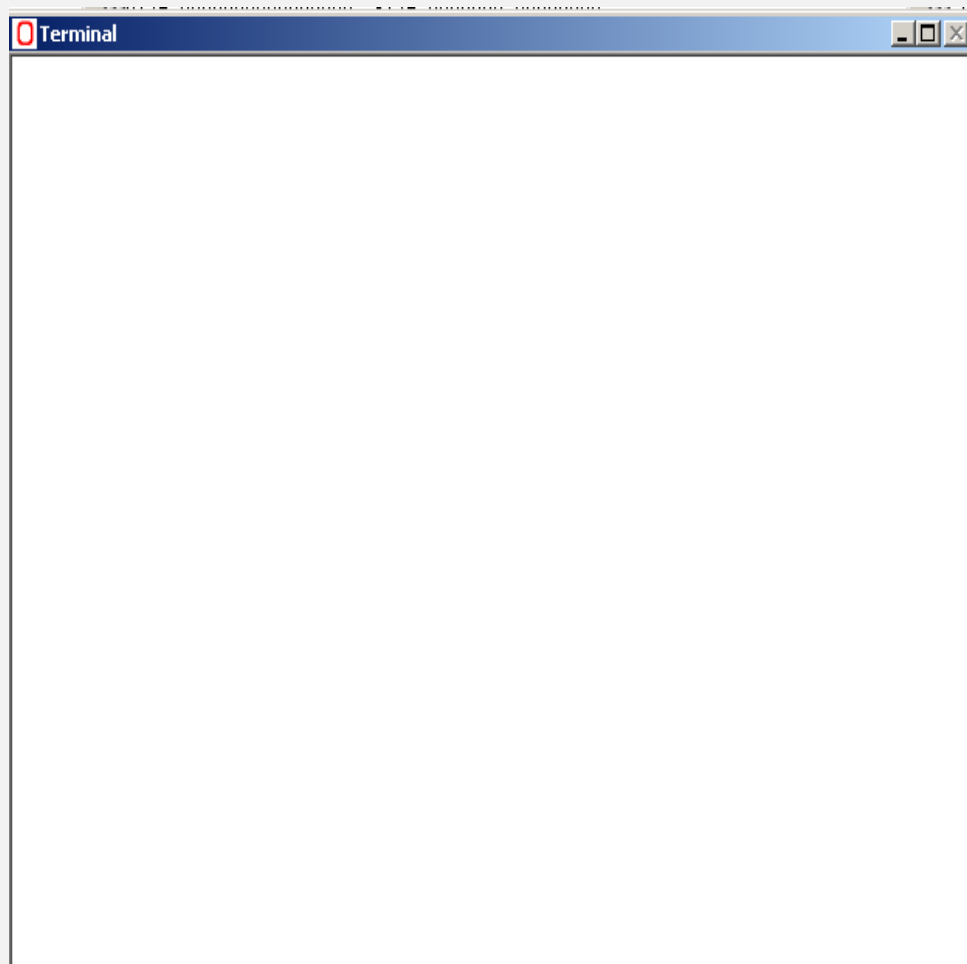
- Code Address Bus: 10
- Data Address Bus: 10
- FP Addition Latency: 4
- FP Multiplier Latency: 7
- FP Division Latency: 24

At the bottom, there are 'OK' and 'Cancel' buttons. Below the buttons, a warning message reads: 'Warning: This will cause a reset!'.

MIPS (MIPS64) - Interface

Menu **Window** - **Terminal**

Terminal de
interacção com o
utilizador (**entrada e
saída de dados**)



MIPS (MIPS64) - Interface

Edite um ficheiro de texto nome sum.s e coloque lá este código

```
                .data
A:              .word 10
B:              .word 8
C:              .word 0

                .text
main:
                ld  r4,A(r0)
                ld  r5,B(r0)
                dadd r3,r4,r5
                sd  r3,C(r0)
                halt
```

MIPS (MIPS64) - Interface

Na opção execute existem várias opções de execução do código

F7 – execução ciclo a ciclo

MIPS (MIPS64) - Interface

Reset

Volta ao início do código, mas não apaga a memória

Reload

Volta ao início do código e apaga a memória

MIPS (MIPS64) - Interface

```

        .data
A:      .word 10
B:      .word 8
C:      .word 0
CR:     .word32 0x10000
DR:     .word32 0x10008

        .text
main:
    ld r4,A(r0)
    ld r5,B(r0)
    dadd r3,r4,r5
    sd r3,C(r0)

    lwu r1,CR(r0)    ;Control Register
    lwu r2,DR(r0)    ;Data Register
    daddi r10,r0,1
    sd r3,(r2)        ;r3 output..
    sd r10,(r1)        ;.. to screen

    halt

```

MIPS (MIPS64) - Interface

- `.data` - start of data segment
- `.text` - start of code segment
- `.code` - start of code segment (same as `.text`)
- `.org <n>` - start address
- `.space <n>` - leave n empty bytes
- `.asciiz <s>` - enters zero terminated ascii string
- `.ascii <s>` - enter ascii string
- `.align <n>` - align to n-byte boundary
- `.word <n1>,<n2>..` - enters word(s) of data (64-bits)
- `.byte <n1>,<n2>..` - enter bytes
- `.word32 <n1>,<n2>..` - enters 32 bit number(s)
- `.word16 <n1>,<n2>..` - enters 16 bit number(s)
- `.double <n1>,<n2>..` - enters floating-point number(s)

MIPS (MIPS64) - Interface

```
.text
```

```
add.d f7,f7,f3
```

```
add.d f7,f7,f4
```

```
mul.d f4,f5,f6 ; WAR on f4
```

MIPS (MIPS64) - Assembler

Case sensitive:

- diretivas (ex: .word, .data)
- registos (ex: \$ra, \$zero)
- variáveis
- etiquetas

NOT case sensitive:

- registos (ex: r1, R1)
- instruções (ex: dadd; DADD)
- hexadecimal (ex: 0xba7; 0xBA7)

Case Sensitive		Not Case Sensitive	
Item	Exemplo	Item	Exemplo
Convenção de registos	\$ra, \$zero	Registos	R1, r1
Diretivas	.data, .word	instruções	daddi, DADDI
Etiquetas	NaoFaz:, Soma:		
Variáveis	DR:, a:, Nome:		

MIPS (MIPS64) - Assembler

Tamanho dos Dados Expressos em bits:

Tamanho dos dados em Bits

Inteiros

Arquitetura	MIPS32	MIPS64
R1	32	64
.word	32	64
.word32	---	32
.word16	---	16
.half	16	---
.byte		8
LD	---	64
LW		32
LH		16
LB		8

Vírgula Flutuante

Single Precision: 32 bits; Double Precision: 64 bits

Arquitetura	MIPS32	MIPS64
F1	32	64
D0 (F0:F1)	64	---
.float	32	32 *
.double		64
L.D		64
L.S		32

* Não implementado no WinMIPS64

MIPS (MIPS64) - Assembler

• Tabela de Instruções e Diretivas MIPS64

Instrução	Exemplo	Significado
Add	dadd R1, R2, R3	$R1 = R2 + R3$
Add Immediate	daddi R1, R2, #100	$R1 = R2 + 100$
Subtract	dsub R1, R2, R3	$R1 = R2 - R3$
And Logical Immediate	andi R1, R2, #20	$R1 = R2 \text{ AND } 20$
Or Logical	or R1, R2, R3	$R1 = R2 \text{ OR } R3$
Load (ld, lw, lb) d=64 bits w=32 bits b=8 bits	ld R1, A(R0)	$R1 = A$
	ld R1, A(R2)	$R1 = \text{Memória} [\text{Endereço de } A + R2]$
	ld R1, (R2)	$R1 = \text{Memória} [0 + R2]$
	ld R1, 200(R2)	$R1 = \text{Memória} [200 + R2]$
Store (ld, lw, lb)	sd R1, 200(R2)	$\text{Memória} [200 + R2] = R1$
Doubleword Shift Left Logical	dsl R1, R2, #3	$R1 = R2 \ll 3 \text{ (} R1 = R2 * 2^3 \text{)}$
Doubleword Shift Right Logical	dslr R1, R2, #4	$R1 = R2 \gg 4 \text{ (} R1 = R2 / 2^4 \text{)}$
Doubleword Shift Left Logical Variable	dslv R1, R2, R3	$R1 = R2 \ll R3 \text{ (} R1 = R2 * 2^{R3} \text{)}$
Set Less Than	slt R1, R2, R3	Se $R2 < R3$ faça $R1 = 1$ senão $R1 = 0$
Set Less Than Immediate	slti R1, R2, #20	Se $R2 < 20$ faça $R1 = 1$ senão $R1 = 0$
Load Floating-point (64 bits)	ld F1, 100(R1)	$F1 = \text{memória} [R1 + 100]$
Store Floating-point (64 bits)	sd F2, 100(R1)	$\text{Memória} [R1 + 100] = F2 \text{ (64 bits)}$
Add Double Precision	add.d F1, F2, F3	$F1 = F2 + F3 \text{ (precisão dupla, 64 bits)}$
Subtract Single Precision	sub.s F1, F2, F3	$F1 = F2 - F3 \text{ (precisão simples, 32 bits)}$
Multiply Double Precision	mul.d F1, F2, F3	$F1 = F2 * F3 \text{ (precisão dupla)}$
Divide Single Precision	div.s F1, F2, F3	$F1 = F2 / F3 \text{ (precisão simples)}$
Move Floating-point	mov.d F1, F2	$F1 = F2 \text{ (precisão dupla)}$
Halt	halt	Para a execução do programa
No Operation	nop	Não faz nada

MIPS (MIPS64) - Assembler

• Tabela de Instruções e Diretivas MIPS64

Saltos condicionais		Salta para etiqueta se ...
Branch Equal Zero	beqz R1, etiqueta	... se R1 = 0 (mesmo que beq R1,R0, etiqueta)
Branch on Not Equal Zero	bnez R1, etiqueta	... se R1 != 0 (mesmo que bne R1,R0, etiqueta)
Branch Equal	beq R1, R2, etiqueta	... se R1 = R2
Branch on Not Equal	bne R1, R2, etiqueta	... se R1 != R2
Saltos incondicionais		Salta para ...
Jump	j etiqueta	... etiqueta
Jump Register	jr R31	... a instrução no endereço guardado em R31
Jump and Link	jal etiqueta	... etiqueta e R31 = PC + 4 (chamada de procedimento)

Outras instruções:

daddu - add integers unsigned	daddui - add immediate unsigned
lbu - load byte unsigned	dsubu - subtract integers unsigned
lh - load 16-bit half-word	sltu - set if less than unsigned
lhu - load 16-bit half word unsigned	sltiu - set if less than or equal imm. unsigned
sh - store 16-bit half-word	dsra - shift right arithmetic
lwu - load 32-bit word unsigned	dsrlv - shift right logical by variable
lui - load upper half of register immediate	dsrav - shift right arithmetic by variable
movz - move if register equals zero	and - logical and
movn - move if register not equal to zero	xor - logical xor
cvt.w.d - convert 32-bit integer to floating-point	ori - logical or immediate
cvt.l.d - convert 64-bit integer to floating-point	xori - exclusive or immediate
mtc1 - move 32 bit integer from integer register to floating-point register	xori - logical xor immediate

MIPS (MIPS64) - Assembler

• Tabela de Instruções e Diretivas MIPS64

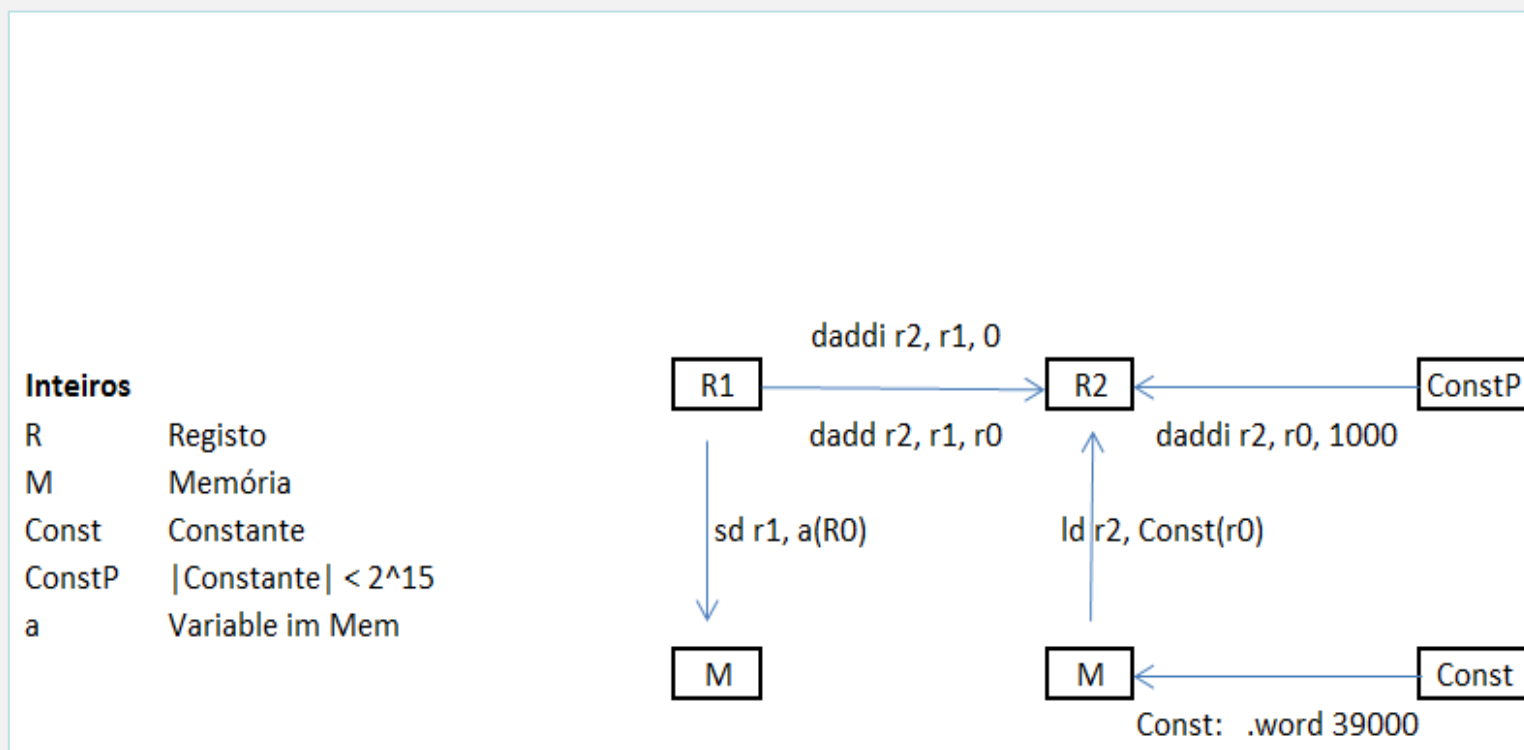
Diretivas para o assembler:

.data	- início do segmento de dados
.text	- início do segmento de programa
.code	- o mesmo que .text
.org <n>	- endereço inicial
.space <n>	- reserva um espaço de n bytes na memória
.asciiz <s>	- define uma cadeia de caracteres terminada por nulo.
.ascii <s>	- define uma cadeia de caracteres
.align <n>	- alinha o próximo endereço para uma fronteira de n-bytes
.word <n1>,<n2>..	- define palavras de 64 bits de dados com valor inicial
.byte <n1>,<n2>..	- define uma sequência de bytes com valor inicial.
.word32 <n1>,<n2>..	- define palavras de 32 bits com valor inicial.
.word16 <n1>,<n2>..	- define palavras de 16 bits com valor inicial.
.double <f1>,<f2>..	- define palavras no formato ponto-flutuante de 64 bits.

Onde <n> denota um número inteiro, <s> significa uma cadeia de caracteres entre aspas e <n1>, <n2> são valores inteiros separados por vírgula e <f1>, <f2> são valores reais separados por vírgula.

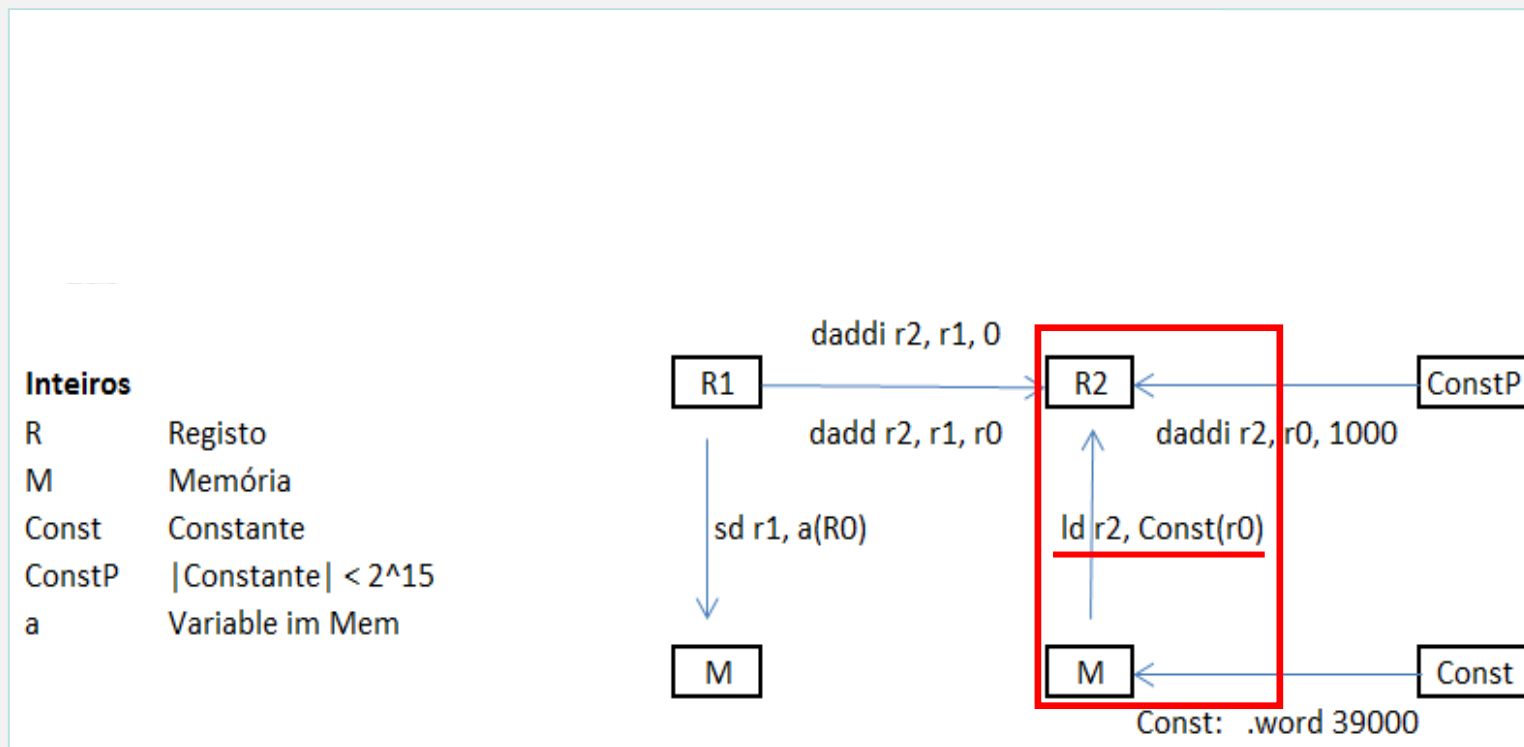
MIPS (MIPS64) - Assembler

- Resumo das opções para a Movimentação de Dados entre Memória - Registos:



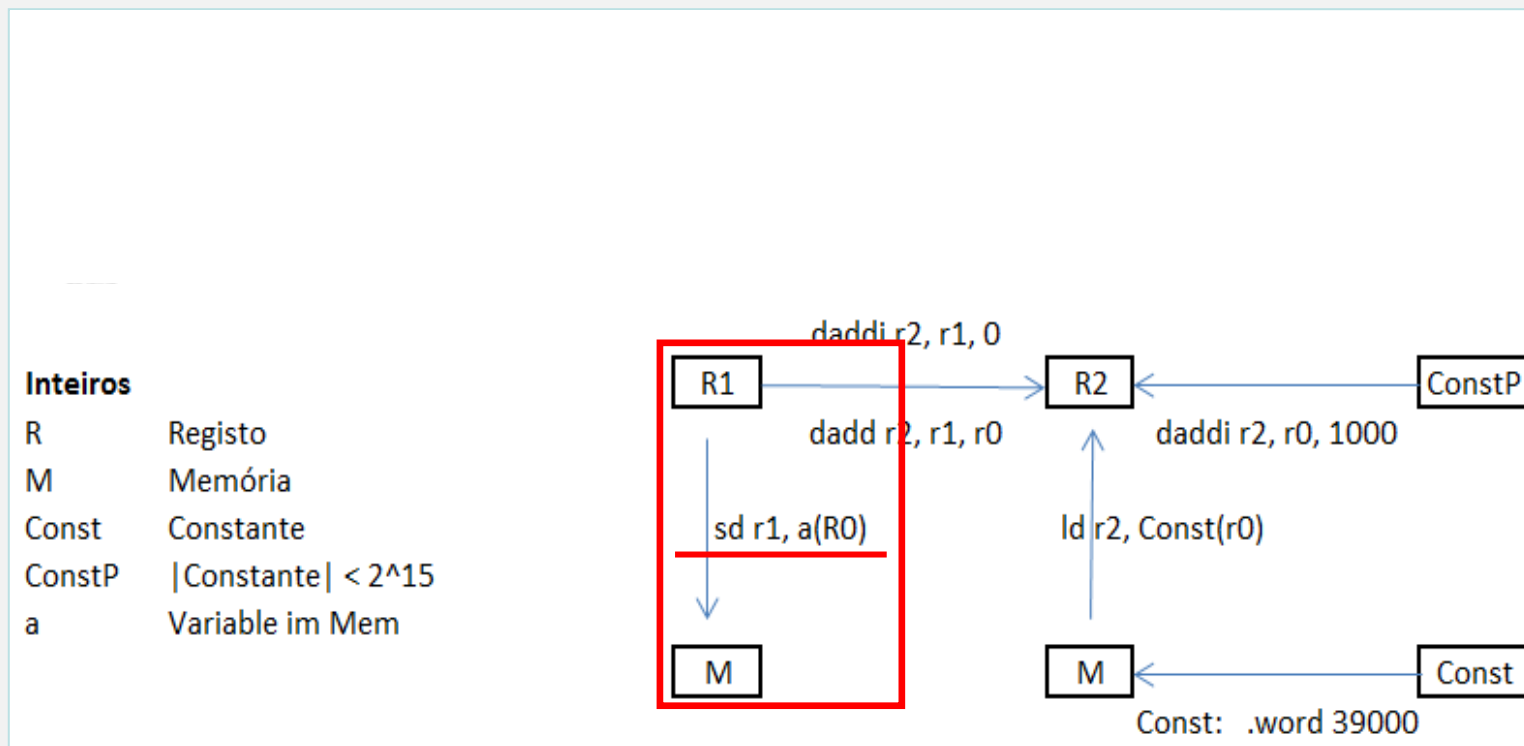
MIPS (MIPS64) - Assembler

- Procedimento para carregar valor da **Memória** (variável) para **Registo**:



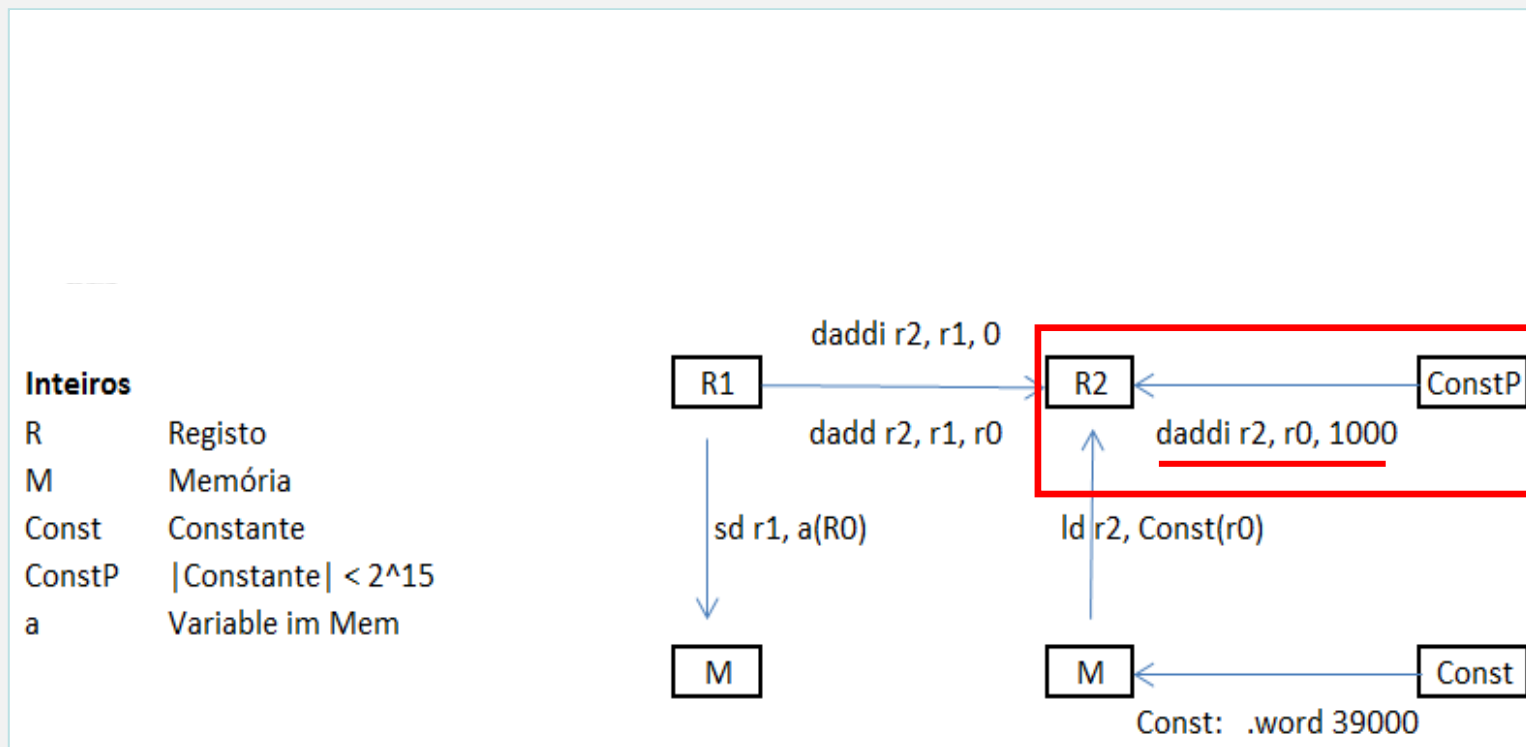
MIPS (MIPS64) - Assembler

- Procedimento para carregar valor do **Registo** para a **Memória**:



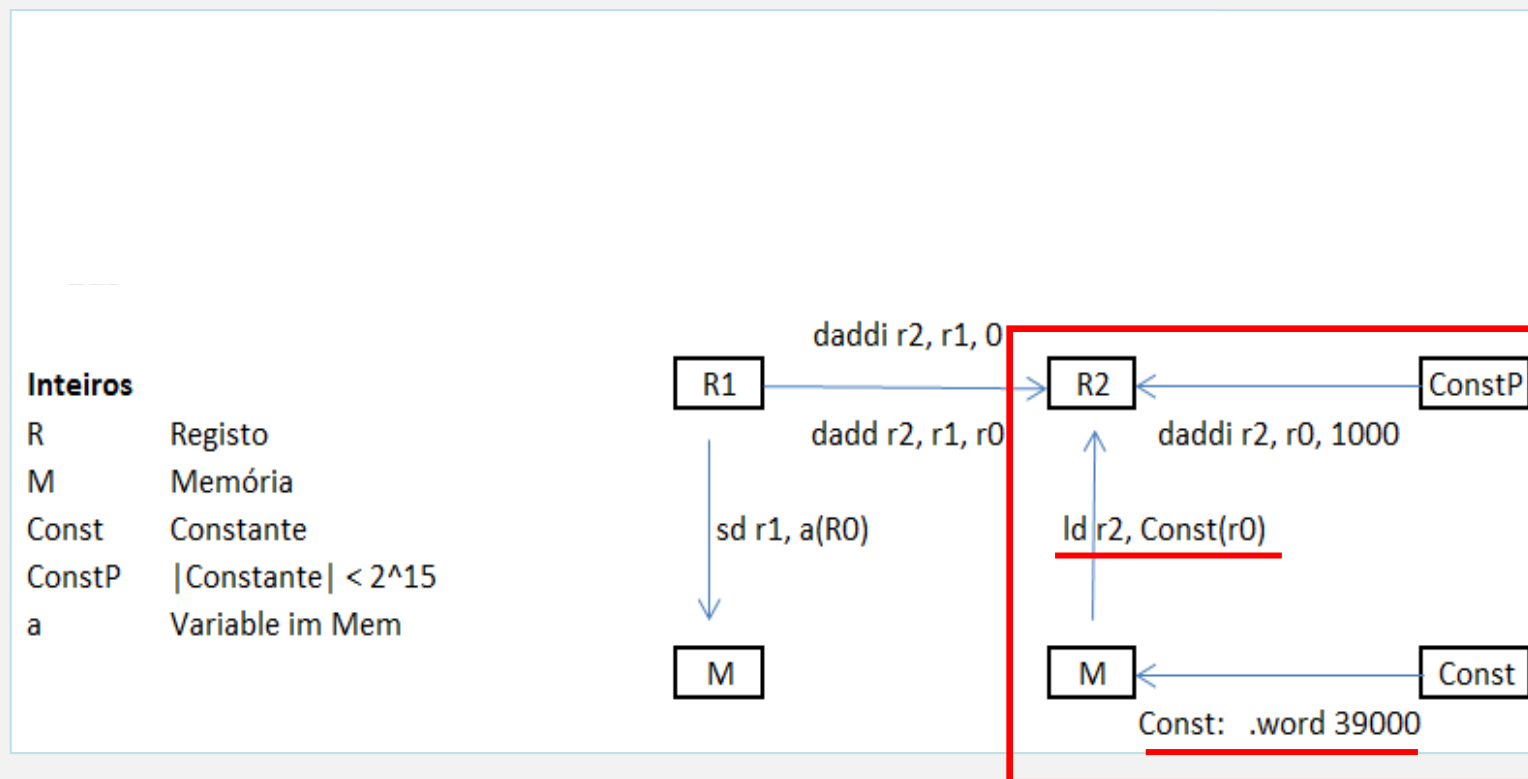
MIPS (MIPS64) - Assembler

- Procedimento colocar **Constante** com “valor pequeno” num **Registo**:



MIPS (MIPS64) - Assembler

- Procedimento colocar qualquer **Constante** num **Registo**:



MIPS (MIPS64) - Assembler

- Memory Mapped I/O Area:

Memory Mapped I/O area

Addresses of CONTROL and DATA registers

CONTROL: .word32 0x10000

DATA: .word32 0x10008

Set CONTROL = 1, Set DATA to Unsigned Integer to be output

Set CONTROL = 2, Set DATA to Signed Integer to be output

Set CONTROL = 3, Set DATA to Floating Point to be output

Set CONTROL = 4, Set DATA to address of string to be output

Set CONTROL = 5, Set DATA+5 to x coordinate, DATA+4 to y coordinate, and DATA to RGB colour to be output

Set CONTROL = 6, Clears the terminal screen

Set CONTROL = 7, Clears the graphics screen

Set CONTROL = 8, read the DATA (either an integer or a floating-point) from the keyboard

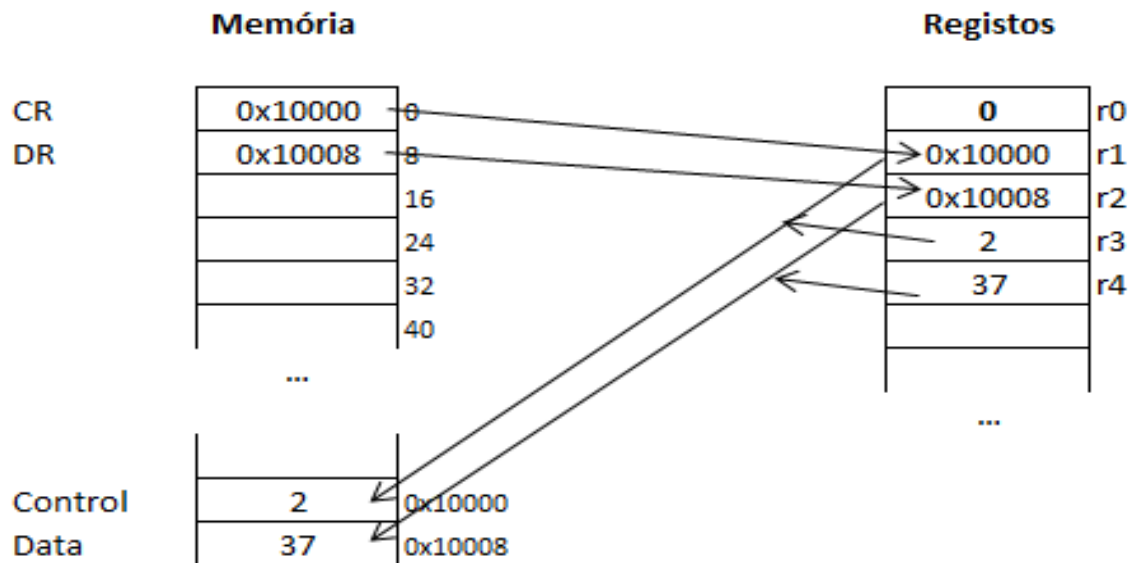
Set CONTROL = 9, read one byte from DATA, no character echo.

MIPS (MIPS64) - Assembler

- Memory Mapped I/O Area:*

- Mostrar valor inteiro na janela “terminal” (enviar para écran)

Signed Integer output to screen (Mostrar r4)



```
.data
CR: .word32 0x10000 ; 0x é para indicar qu
DR: .word32 0x10008 ; 0x é para indicar qu
;r1:CR r2:DR r3:aux r4:aux
```

```
.code
lwu r1, CR(r0) ; Control Register
lwu r2, DR(r0) ; Data Register

daddi r4,r0,37 ; enviar o valor 37 para DATA
sd r4,(r2)

daddi r3,r0,2 ; enviar código 2 para CODE (código para inteiro com sinal)
sd r3,(r1) ; chama rotina
halt
```

Set CONTROL = 2, Set DATA to Signed Integer to be output

MIPS (MIPS64) - Assembler

- *Memory Mapped I/O Area:*
 - Obter valor inteiro (ou de virgula flutuante) do teclado

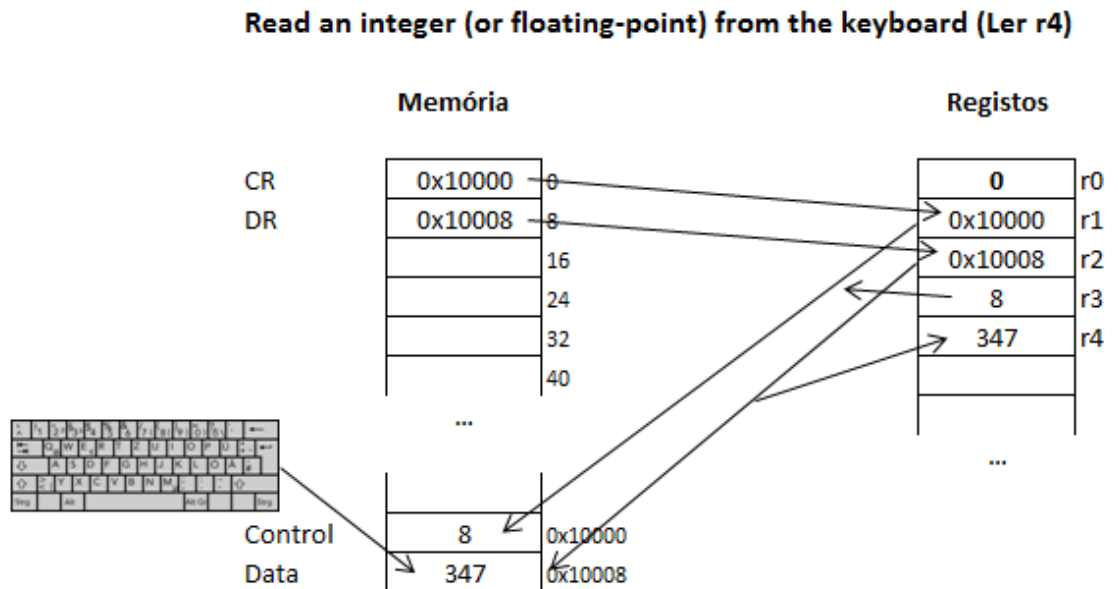
```
.data
CR: .word32 0x10000 ; 0x é para indicar que é numero hexadecimal
DR: .word32 0x10008 ; 0x é para indicar que é numero hexadecimal
;r1:CR r2:DR r3:aux r4:aux
```

```
.code
lwu r1,CR(r0) ;Control Register
lwu r2,DR(r0) ;Data Register
```

```
daddi r3,r0,8 ; Enviar 8 para CODE (código para ler dados de teclado)
sd r3,(r1) ; Chama rotina para a introdução de dados (usando teclado) em DATA

ld r4,(r2) ; Valor introduzido pelo teclado (ex: 347) que fica guardado em
; DATA é colocado em r4 por ex.

halt
```



Set CONTROL ← 8, read the DATA (either an integer or a floating-point) from the keyboard

MIPS (MIPS64) - Assembler

- Obter inteiro (ou virgula flutuante) de teclado (CODE 8) :

- Mostrar inteiro s/ sinal (CODE 1) no écran:

```
.data
CR: .word32 0x10000 ; 0x é para indicar que é numero hexadecimal
DR: .word32 0x10008 ; 0x é para indicar que é numero hexadecimal
;r1:CR r2:DR r3:aux r4:aux
```

```
.code
lwu r1, CR(r0) ; Control Register
lwu r2, DR(r0) ; Data Register

daddi r3,r0,8 ; enviar código 8 para CODE
; (código para obter dados de teclado)

sd r3,(r1) ; chama rotina para obter valor do teclado
ld r4,(r2) ; valor obtido que fica em DATA é
; colocado em r4
halt
```

```
.code
lwu r1, CR(r0) ; Control Register
lwu r2, DR(r0) ; Data Register

daddi r4,r0,37 ; por ex. 37 que é para ser
; mostrado é colocado em r4
daddi r3,r0,1 ; enviar código 2 para CODE
; (código p/ inteiro s/sinal)

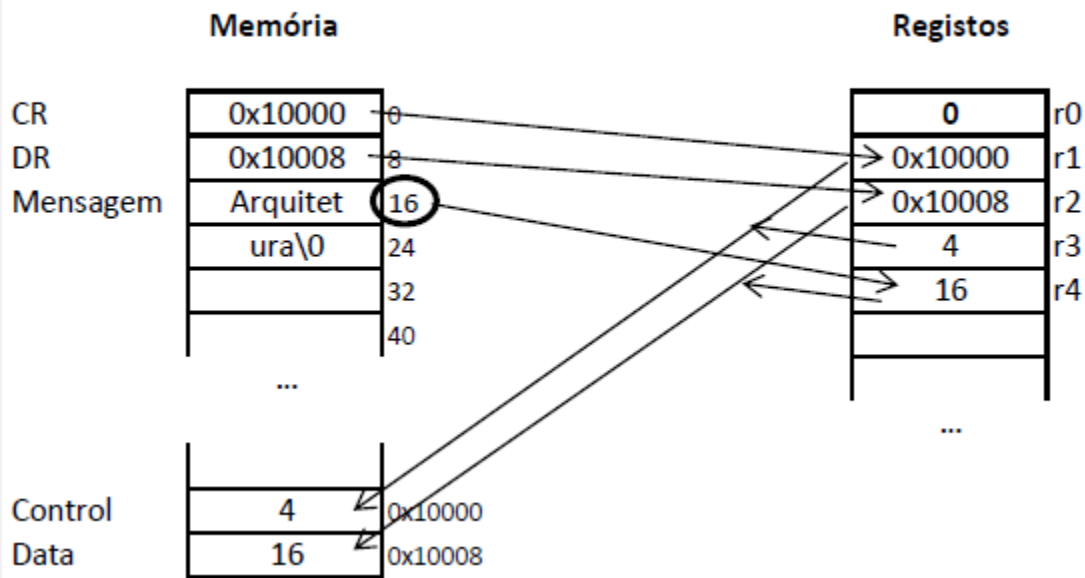
sd r4,(r2) ; valor 37 para DATA
sd r3,(r1) ; chama rotina para mostrar valor
halt
```

MIPS (MIPS64) - Assembler

- Memory Mapped I/O Area:*

- Mostrar *string* no écran

String output to screen (Mostrar Mensagem)



```
.data
CR:          .word32 0x10000
DR:          .word32 0x10008
Mensagem:    .asciiz "Arquitetura"
```

```
.text
lwu r1,CR(r0) ;Control Register
lwu r2,DR(r0) ;Data Register
```

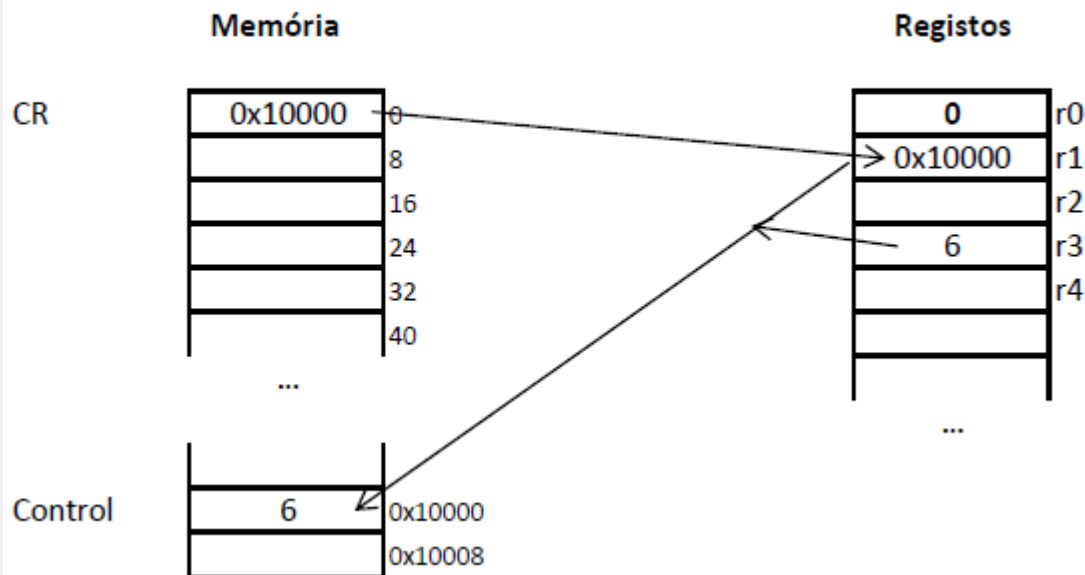
```
daddi r3,r0,Mensagem
sd r3,(r2)
daddi r3,r0,4
sd r3,(r1)
```

Set CONTROL = 4 Set DATA to address of string to be output

MIPS (MIPS64) - Assembler

- *Memory Mapped I/O Area:*
 - Limpar écran

Clears the terminal screen (Apaga ecrã)



```
.data
CR:      .word32 0x10000

.text
lwu r1,CR(r0) ;Control Register

daddi r3,r0,6
sd r3,(r1)
```

Set CONTROL = 6 Clears the terminal screen

Créditos

Créditos

- Nuno Veiga, Rui Vasco Monteiro